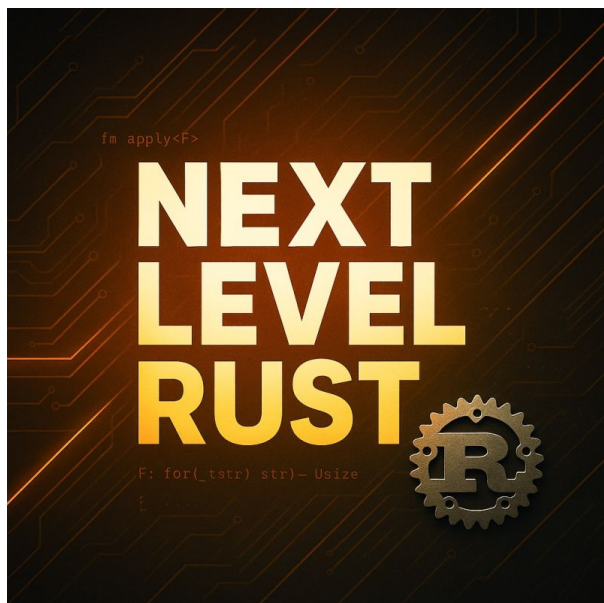




Copyrighted material

This is an integral part of course **Next level Rust** and cannot be used in any context without explicit permission from **CLEUTON SAMPAIO DE MELO JUNIOR** (a.k.a. "the Author"). You can't copy, change, publish, or share with others.

What You Need to "Unlearn"

Clean Code the Rust Way

Why this title? The Rust Way because not all traditional clean code or SOLID rules apply directly to Rust.

Those guidelines were created for object-oriented languages like Java or C#. Rust is different. Its type system, ownership model, and trait system lead to other practices.

For example:

- Instead of forcing small classes or inheritance hierarchies, Rust favors small, focused functions and composition through traits.
- Instead of relying on runtime checks and exceptions, Rust encodes safety in the type system, making invalid states impossible to create.
- Instead of implicit behavior, Rust makes ownership, borrowing, and error handling explicit in the code.
- Instead of garbage collection, Rust guarantees deterministic cleanup at scope boundaries.

So The Rust Way means adapting the spirit of clean code to Rust's reality. We still value clarity, safety, and maintainability. But we achieve them with smaller functions, explicit types, compile-time guarantees, and composition over inheritance. These principles are a practical summary of what clean code looks like when written in Rust.

Demo project

This project is a small order processor. It takes raw text input, validates it, turns it into structured domain data, stores it, logs the steps, and demonstrates deterministic cleanup. The goal is to show four clean code principles in practice:

- Less is more
- Explicit over implicit
- Compile-time safety over runtime checks
- Composition over inheritance

Less is more: small and focused functions

Instead of one large function that does everything, the system uses small steps. Parsing, validation, and transformation are separate functions:

```
pub fn parse_order(input: &str) -> Result<ParsedOrder<'_>, ParseError> { /*  
... */ }  
  
pub fn validate_order(p: ParsedOrder<'_>)  
-> Result<ValidOrder<'_>, ValidationError> { /* ... */ }  
  
pub fn parse_lines(lines: &str, currency: Currency)  
-> Result<Vec<OrderLineOwned>, ParseError> { /* ... */ }
```

These are composed in the service layer:

```
pub fn place_order(&mut self, input: &str) -> Result<Order,  
ProcessingError> {  
    let p = parse_order(input)?;  
    let v = validate_order(p)?;  
    let lines = parse_lines(v.lines, v.currency)?;  
    let order = Order { id: v.id, customer_id: v.customer_id,  
                        created_at_epoch_s: chrono::Utc::now().timestamp(),  
                        currency: v.currency, lines };  
    self.repo.insert(order.clone())?;  
    self.notifier.notify("order placed");  
    Ok(order)  
}
```

Each function has one clear job. This makes testing easier, logs more meaningful, and the codebase easier to maintain.

Explicit over implicit: make intent clear

The code avoids hidden behavior. Borrowed vs. owned data is always visible:

```
pub struct ParsedOrder<'a> {
    pub id: &'a str,
    pub customer_id: &'a str,
    pub currency: &'a str,
    pub lines: &'a str,
}

pub struct OrderLineOwned {
    pub sku: NonEmptyString, // owns the string
    pub qty: PositiveQuantity, // validated
    pub unit: Money, // has currency and value
}
```

While parsing, the program uses `&str` slices, with no allocations. When building domain data that must live independently, it converts into owned types like `String`. The moment of allocation is explicit.

Failures and optional values are also explicit. Instead of exceptions or nulls, every fallible operation returns a `Result`:

```
pub fn parse_order(...) -> Result<ParsedOrder<'_>, ParseError>;
pub fn validate_order(...) -> Result<ValidOrder<'_>, ValidationError>;
```

Dependencies are declared in type signatures. The service shows exactly what it needs:

```
pub struct OrderService<R: OrderRepository, N: Notifier, L: AuditLogger> {
    repo: R,
    notifier: N,
    logger: L,
}
```

There is no hidden global state. Both compiler and developers know the contracts.

Compile-time safety over runtime checks

The project encodes rules in the type system so that invalid states cannot exist. Quantities and currencies are validated at creation:

```
pub struct PositiveQuantity(u32);
impl PositiveQuantity {
    pub fn new(v: u32) -> Result<Self, ValidationError> {
        if v == 0 { Err(ValidationError::NonPositiveQuantity) }
        else { Ok(Self(v)) }
    }
}
```

```
pub enum Currency { BRL, USD, EUR }

pub struct Money { pub cents: i64, pub currency: Currency }
impl Money {
    pub fn add(self, other: Money) -> Result<Money, MoneyError> {
        if self.currency != other.currency {
            Err(MoneyError::CurrencyMismatch)
        } else {
            Ok(Money { cents: self.cents + other.cents, currency:
self.currency })
        }
    }
}
```

An order's total is safe by construction:

```
pub fn total(&self) -> Result<Money, MoneyError> {
    let mut acc = Money::zero(self.currency);
    for l in &self.lines {
        let line_total = Money {
            cents: l.unit.cents * l.qty.get() as i64,
            currency: self.currency,
        };
        acc = acc.add(line_total)?;
    }
    Ok(acc)
}
```

It is impossible to create a zero quantity or to sum values in different currencies without the compiler raising an error.

Composition over inheritance

Instead of class hierarchies, the project uses traits. Each trait models one behavior:

```
pub trait Notifier { fn notify(&self, msg: &str); }
pub trait AuditLogger { fn log(&self, msg: &str); }

pub struct StdoutNotifier;
pub struct StdoutLogger;

impl Notifier for StdoutNotifier {
    fn notify(&self, msg: &str) { println!("[notify] {msg}"); }
}
impl AuditLogger for StdoutLogger {
    fn log(&self, msg: &str) { println!("[audit] {msg}"); }
}
```

The service is generic and can work with any notifier or logger:

```
pub struct OrderService<R: OrderRepository, N: Notifier, L: AuditLogger> {  
    repo: R,  
    notifier: N,  
    logger: L,  
}
```

Functionality grows by combining small behaviors. You can swap `StdoutNotifier` for an `EmailNotifier` without touching the service logic.

Deterministic cleanup

Rust ensures resources are freed exactly at the end of scope. This is visible in the `ExportBuffer` example:

```
pub struct ExportBuffer { name: &'static str, buf: Vec<u8> }  
  
impl Drop for ExportBuffer {  
    fn drop(&mut self) { println!("free {} (via Drop)", self.name); }  
}
```

Running the demo shows `export-B` freed first when the inner block ends, and `export-A` freed later at the outer block. Cleanup is precise and guaranteed, unlike in runtimes with garbage collection.

Summary

- **Less is more:** parsing, validation, and transformation are separate, tiny functions composed in the service.
- **Explicit over implicit:** types show ownership, errors, and dependencies clearly.
- **Compile-time safety:** invariants like positive quantities and matching currencies are enforced at construction.
- **Composition over inheritance:** behaviors like logging and notification are added via traits, not rigid hierarchies.
- **Deterministic cleanup:** Rust drops resources exactly at scope end, with no GC pauses.

This makes the codebase safer, clearer, and easier to extend.